EAST WEST UNIVERSITY

CSE246: Algorithms (Section No. 02)
[Fall 2025]

Group No: 05

Project Report

Graph Route Bangladesh

Submitted by:

GROUP MEMBERS	STUDENTS ID
Md.Sabik Hossen	2023-2-60-305
Mahadi Prodhan	2023-2-60-033
Hadiul Alam Hredoy	2023-2-60-284
Mohammad Tanweer Islam	2022-3-60-199

Submitted to:

Dr. Md. Tauhid Bin Iqbal

Assistant Professor

Department of CSE, East West University

Date of Submission : 17th December 2025

`

Table of Content

1. Introduction 1

1.1 Problem Statement 1

1.2 Motivation and Objectives of The Project 1

2. Literature Review 2

2.1 Relevant Theories, Methods 2

2.2 Limitations in Existing Approaches 3

3. Proposed Method 4

3.1 System Design / Methodology 4

3.2 Algorithms, Models Used 5

3.3 Tool, Technologies, or Datasets Applied 6

4. Implementation 7

4.1 Description for Project Modules 7

4.2 Key Functions, Workflow 12

5. Results and Discussion 15

5.1 Output Screenshot / Performance Analysis 15

5.2 Comparison Results 19

5.3 Key Findings 20

6. Conclusion and Future Work 21

6.1 Summary of Contributions 21

6.2 Limitations and Potential Future Improvements 21

`

1. Introduction:

1.1 Problem Statement:

Bangladesh has a complex and interconnected transportation network made up of districts, highways, and regional roads. Finding the shortest and most efficient route between two locations is a major challenge for travelers, logistics services, emergency responders, and transportation planners. Traditional methods of planning routes can be slow, prone to mistakes, and inefficient, especially when dealing with multiple routes and distances. The task is to model the districts of Bangladesh as a graph. Each district will be a node, while the roads connecting them will be edges with distances or costs attached. By using graph theory, the system needs to calculate the shortest path between any two districts quickly and accurately. This project plans to design and implement a route analysis system based on Graph Theory. It will use established shortest path algorithms like Dijkstra's Algorithm, Bellman-Ford Algorithm, and Floyd-Warshall Algorithm. The system will let users enter a starting district and a destination district, then provide the best route and the total distance.

1.2 Motivation and Objectives of The Project:

To apply graph theory algorithms for efficiently finding shortest routes between districts in Bangladesh and demonstrate their real-world relevance.

Objectives:

- Model Bangladesh's districts as a graph
- Apply shortest path algorithms
- Compute optimal routes efficiently
- Enhance practical understanding of graph theory

2. Literature Review:

2.1 Relevant Theories, Methods:

- **Vertices (Nodes):** Districts of Bangladesh
- **Edges:** Roads connecting districts
- **Weights:** Distance or travel cost between districts

`

Dijkstra's Algorithm:

Dijkstra's Algorithm is a greedy algorithm used to find the shortest path from a single source node to all other nodes in a graph with non-negative edge weights.

Method:

- Initialize all distances as infinity except the source.
- Use a priority queue to select the nearest unvisited node.
- Relax edges to update shortest distances.

■ **Implementation Strategy:**

Key Components:

■ **Priority queue (min-heap)** for selecting the minimum-distance vertex

■ **Distance array** initialized to infinity for all vertices

■ **Parent array** to track the path taken

■ **Relaxation process** to update distances when a shorter route is found

■ **Visited check** to avoid reprocessing vertices

Time Complexity : O((V + E) log V) using binary heap

Space Complexity: O(V)

Pseudocode:

```
pair<vector<int>, vector<int>> dijkstra(const Graph &G, int src) {
    vector<int> dist(G.vertices, INF), parent(G.vertices, -1);
    dist[src] = 0;
    priority_queue<pair<int,int>, vector<pair<int,int>>, greater<pair<int,int>>> pq;
    pq.push({0,src});
    while(!pq.empty()) {
        auto [d,u] = pq.top(); pq.pop();
        if(d > dist[u]) continue;
        for(auto &[v,w] : G.adjacent_list[u]) {
            if(dist[u]+w < dist[v]) {
                dist[v] = dist[u]+w;
                parent[v] = u;
                pq.push({dist[v],v});
            }
        }
    }
    return {dist,parent};
}
```

`

Bellman-Ford Algorithm:

Bellman-Ford is a dynamic programming–based algorithm that computes shortest paths from a single source and supports **negative edge weights**.

Method:

· Iteratively relaxes all edges to find optimal distances.

· Detects negative weight cycles if present.

· Uses an **edge list** representation of the graph.

Implementation Strategy:
Key Components:

- **Distance array** initialized to infinity
- **Parent array** for path reconstruction
- **Edge relaxation repeated (V − 1) times.**
- Handles negative weights safely.
- Can detect **negative weight cycles** (if needed)

Time Complexity : O(V × E)

Space Complexity: O(V)

Pseudocode:

```
pair<vector<int>, vector<int>> bellmanFord(const Graph &G, int src) {
    vector<int> dist(G.vertices, INF), parent(G.vertices, -1);
    dist[src] = 0;
    for(int i=0;i<G.vertices-1;i++)
        for(auto &[u,v,w] : G.edges)
            if(dist[u] != INF && dist[u]+w < dist[v]) { dist[v] = dist[u]+w; parent[v]=u; }
    // Negative cycle check
    for(auto &[u,v,w] : G.edges)
        if(dist[u]!=INF && dist[u]+w<dist[v]) { cout<<"Negative cycle!\n"; return {{}, {}};
    return {dist,parent};
}
```

`

Floyd-Warshell Algorithm:

The Floyd-Warshall Algorithm is a dynamic programming(DP) algorithm used to find the shortest paths between all pairs of vertices in a weighted graph. It works on both directed and undirected graph. But it not applicable in negative weight, weightless graph. We can change the undirected graph to directed graph like this.

Method :

- Computes shortest paths between **every pair of districts**.
- Uses a 2D distance matrix.
- Suitable for pre-processing and route analysis.

Implementation Strategy:

Key Components:

- **Distance matrix** initialized with direct edge weights
- Diagonal entries automatically set to **0**
- Triple nested loop to evaluate:
  dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j])
- Efficiently updates every possible pairwise route
- Works for negative weights (but not negative cycles)

Time Complexity:$O(V^3)$
Space Complexity:$O(V^2)$

Pseudocode:

```
pair<vector<int>, vector<int>> floydWarshall(const Graph &G, int src) {
    int V = G.vertices;
    vector<vector<int>> dist(V, vector<int>(V, INF));
    vector<vector<int>> next(V, vector<int>(V,-1));
    for(int i=0;i<V;i++) {
        dist[i][i]=0;
        for(auto &[v,w]:G.adjacent_list[i]) { dist[i][v]=w; next[i][v]=v; }
    }
    for(int k=0;k<V;k++)
        for(int i=0;i<V;i++)
            for(int j=0;j<V;j++)
                if(dist[i][k]!=INF && dist[k][j]!=INF && dist[i][k]+dist[k][j]<dist[i][j])
                    dist[i][j]=dist[i][k]+dist[k][j]; next[i][j]=next[i][k];
                }
```

`

2.2 limitations in Existing Approaches:

i. The system relies on predefined and static road distance data. It does not support real-time updates such as traffic congestion, road closures, or accidents.
ii. Current shortest path calculations consider only distance or cost, ignoring real-world factors like traffic density, travel time variation, or peak-hour delays.
iii. Route selection is based on a single weight (distance or cost). Multiple criteria such as time, fuel consumption, and road quality are not considered.
iv. Road and distance data must be entered manually or through files, increasing the risk of data inconsistency or outdated information.

The Benefit of Dijastra's,Bellaman Ford and Floyd Warshell:

I. Dijkstra's algorithm is very fast for graphs with non-negative weights, making it suitable for large road networks.It guarantees the shortest path from a single source to all reachable destinations.Ideal for transportation systems where distances and travel costs are always positive.

II. It can detect negative weight cycles, which is important for validating route data.

Works correctly even when graph conditions are not ideal.Useful for national-level planning and full network evaluation

III. . Computes shortest paths between every pair of nodes in a single execution.Once computed, route queries between any two nodes are instant.

3. Proposed Method:

3.1 System Design / Methodology:

Dijastra's Algorithm

The project follows a modular design approach with clear separation of concerns:

- **cityToIndex : Maps city names (strings) to integer indices for efficient lookup**.
- **indexToCity**: **Maps integer indices back to city names for user-friendly output.**
- **adjacent_list: Represents the graph as an adjacency list, where each node stores a list of connected nodes and edge weights**.
- **edges: Maintains a list of edges for algorithms like Bellman-Ford and Floyd-Warshall that require edge-based iteration.**

Core Functions

`

- **addCity(): Normalize city names (remove spaces + lowercase)**
- **addEdge(): Bidirectional edge creation**
- **loadFromFile(): Data import functionality**

Algorithm Implementations

- **dijkstra(): Single-source shortest path**
- **bellmanFord(): Negative weight handling**
- **floydWarshall(): All-pairs shortest path**

3.2 Algorithms, Models Used:

Dijastra's Algirthm:
It chooses the "locally optimal" node (node with the smallest tentative distance) at each step.

1.Initialize:

- Distance of source node = 0.
- Distance of all other nodes = ∞.
- Mark all nodes as unvisited.

2.Select the unvisited node with the smallest distance (greedy choice).

3. Update the distances of all neighbors of the selected node:

```
if distance[current] + weight(current, neighbor) < distance[neighbor]:

    distance[neighbor] = distance[current] + weight(current, neighbor)
```

BellamanFord Algirthm:

Find shortest paths from a single source to all nodes in a weighted graph (can handle negative weights). Detects negative weight cycles.It **relaxes all edges repeatedly** to propagate shortest distance estimates.

1.**Initialize:**

- Distance[source] = 0
- Distance[all others] = ∞

`

2. Repeat for (V-1) iterations:

if distance[u] + w < distance[v]:

distance[v] = distance[u] + w

3.Check for negative weight cycles:

If any edge can still be relaxed, a negative weight cycle exists.

Floyd-Warshall Algorithm

Find shortest paths between all pairs of vertices in a weighted graph. Handles negative weights but no negative cycles.It builds solutions for larger subproblems using smaller subproblems.

1.Initialize a distance matrix dist[i][j]:

- **dist[i][i] = 0**
- **dist[i][j] = weight(i, j) if edge exists, else ∞**

2.For each intermediate vertex k:

- For each pair of vertices (i, j):

dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j])

This checks if going through k provides a shorter path.

3.3 Tools, Technologies, or Datasets Applied:

Programming Language: C++

Libraries: iostream, fstream, cstring, bits/stdc++.h

File Format:

Project.cpp

Compiler: MinGW, Visual Studio

`

4. Implementation:

4.1 Description for Project Modules:

Dijastra's Algorithm :

```
while(!pq.empty()) {
    auto [d,u] = pq.top(); pq.pop();
    if(d > dist[u]) continue;
```

Process nodes in priority queue for Dijkstra's algorithm. Always expands the node with the smallest current distance and ignores outdated entries to ensure optimal shortest paths.

Dijastra's Algorithm Construction Module:

```
for(auto &[v,w] : G.adjacent_list[u]) {
    if(dist[u]+w < dist[v]) {
        dist[v] = dist[u]+w;
        parent[v] = u;
        pq.push({dist[v],v});
    }
}
```

The algorithm repeatedly extracts the nearest unvisited vertex, relaxes all adjacent edges by updating shorter distances, and stores parent information to reconstruct the shortest path.

Bellaman-Ford Algorithm :

```
for(int i=0;i<G.vertices-1;i++)
    for(auto &[u,v,w] : G.edges)
        if(dist[u] != INF && dist[u]+w < dist[v]) { dist[v] = dist[u]+w; parent[v]=u; }
```

Bellman-Ford Algorithm finds the shortest path from a starting point to all other points in a graph. It works by repeatedly checking every edge (road) and updating distances. Unlike Dijkstra, it can handle negative weights and detect negative cycles.

`

BellamanFord Algorithm Construction Module:

```
for(auto &[u,v,w] : G.edges)
    if(dist[u]!=INF && dist[u]+w<dist[v]) { cout<<"Negative cycle!\n"; return {{}, {}};
}
```

It repeatedly relaxes all edges of the graph for V−1V-1V−1 iterations, ensuring that the shortest distances are propagated across the network. After the relaxation phase, the algorithm performs an additional iteration to detect negative-weight cycles.

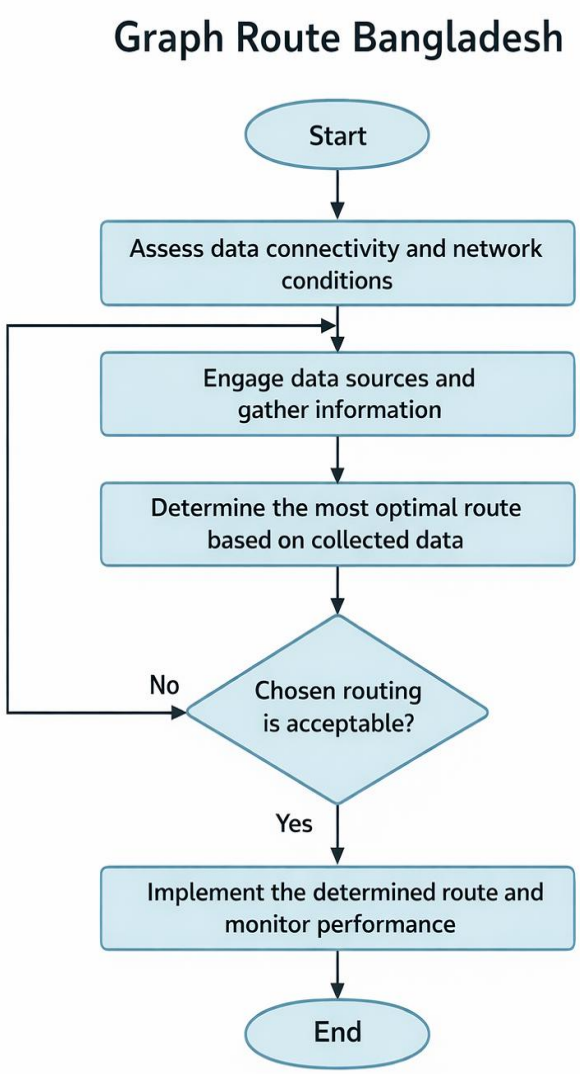
Floyd-Warshell Algorithm :

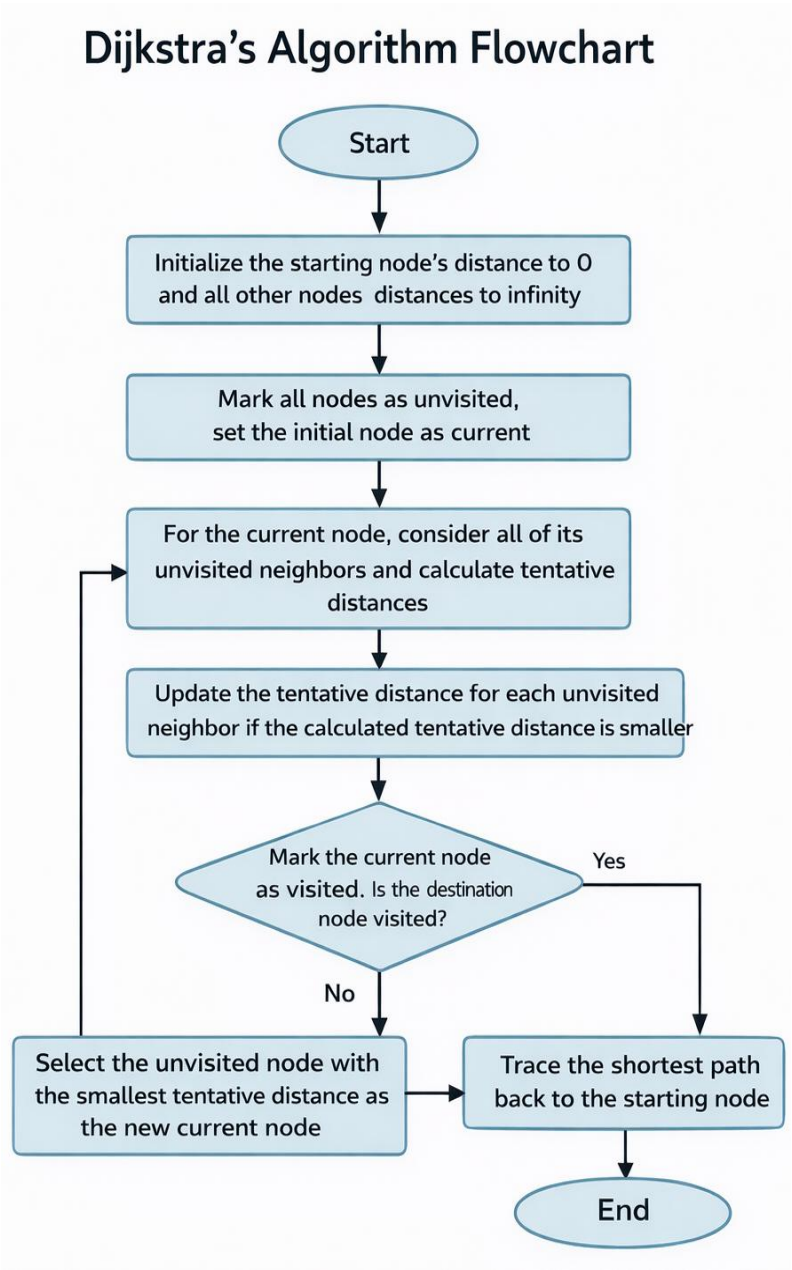
```
for(int k=0;k<V;k++)
    for(int i=0;i<V;i++)
        for(int j=0;j<V;j++)
            if(dist[i][k]!=INF && dist[k][j]!=INF && dist[i][k]+dist[k][j]<dist[i][j]) {
                dist[i][j]=dist[i][k]+dist[k][j]; next[i][j]=next[i][k];
            }
```

Dynamic programming algorithm that computes **all-pairs shortest paths** by progressively improving path estimates.

`

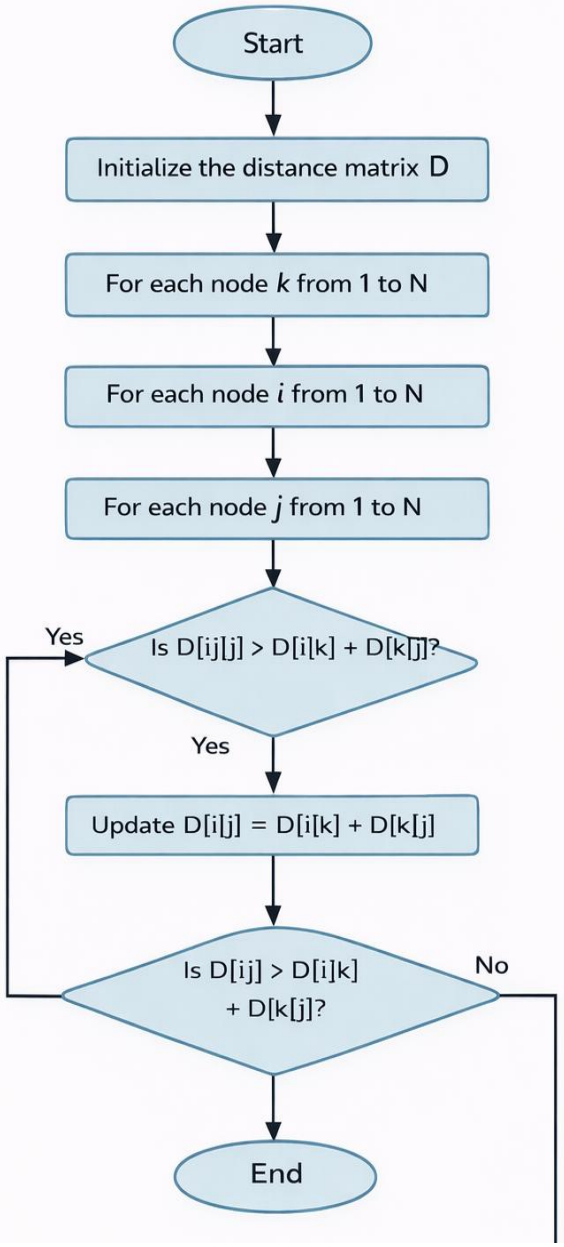
4.2 Key Functions, Workflow:



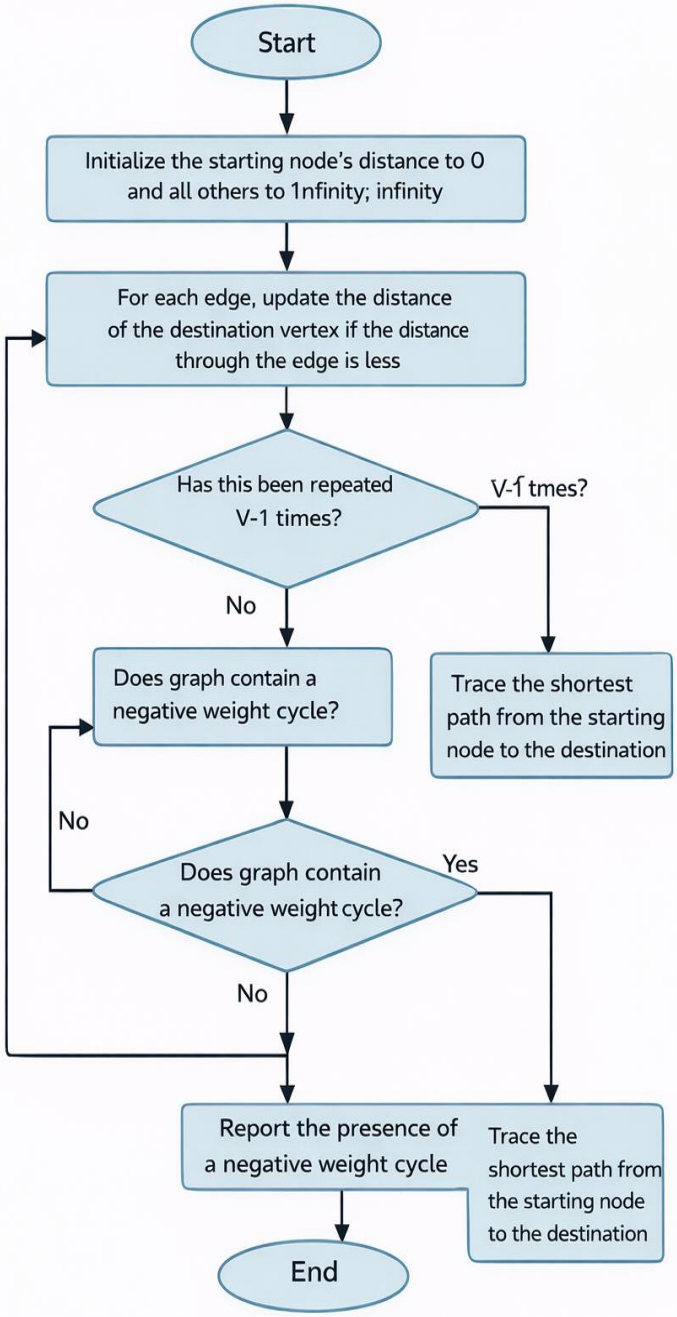


`

Floyd-Warshall Algorithm Flowchart



Bellman-Ford Algorithm Flowchart



`

5. Results and Discussion:

5.1 Output Screenshot / Performance Analysis:

```
-----------------------------------
 Graph Route Bangladesh
-----------------------------------
1. Show all districts
2. Find shortest path (Dijkstra)
3. Find shortest path (Bellman-Ford)
4. Find shortest path (Floyd-Warshall)
5. Exit
Enter choice:
```

This visualization helps users:

- Identify correct district names
- Understand the scope of the network
- Verify spelling and availability

When users select option 1, the project displays all 64 districts in an organized column format:

```
Available Districts (64):
Bagerhat        Bandarban       Barguna         Barisal         Bhola
Bogra           Brahmanbaria    Chandpur        Chittagong      Chuadanga
Coxsbazar       Cumilla         Dhaka           Dinajpur        Faridpur
Feni            Gaibandha       Gazipur         Gopalganj       Habiganj
Jamalpur        Jessore         Jhalokati       Jhenaidah       Joypurhat
Khagrachari     Khulna          Kishoreganj     Kurigram        Kushtia
Lakshmipur      Lalmonirhat     Madaripur       Magura          Manikganj
Meherpur        Moulvibazar     Munshiganj      Mymensingh      Naogaon
Narail          Narayanganj     Narsingdi       Natore          Nawabganj
Netrokona       Nilphamari      Noakhali        Pabna           Panchagarh
Patuakhali      Pirojpur        Rajbari         Rajshahi        Rangamati
Rangpur         Satkhira        Shariatpur      Sherpur         Sirajganj
Sunamganj       Sylhet          Tangail         Thakurgaon
```

Input Query:

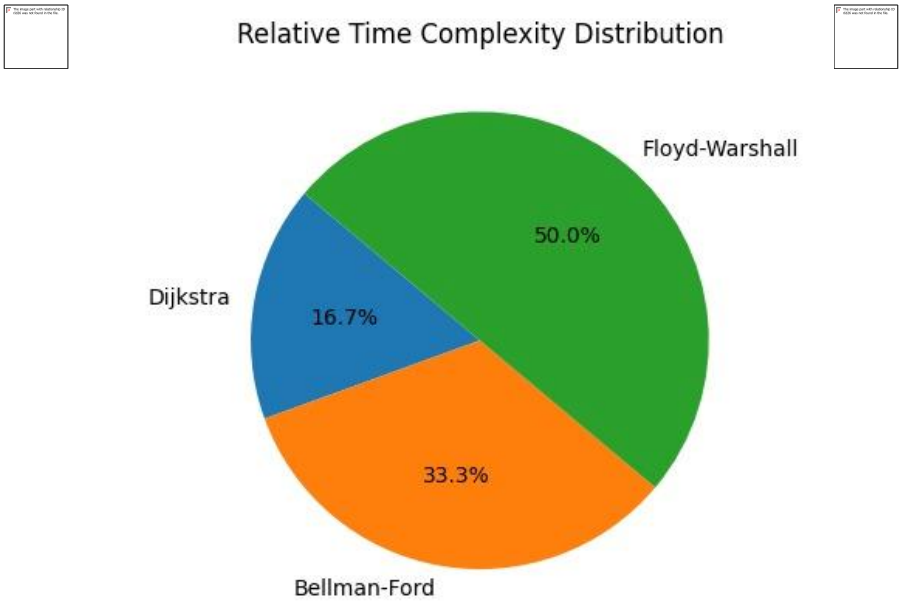
```
Enter choice: 02
Source : Dhaka
Destination : Faridpur
```

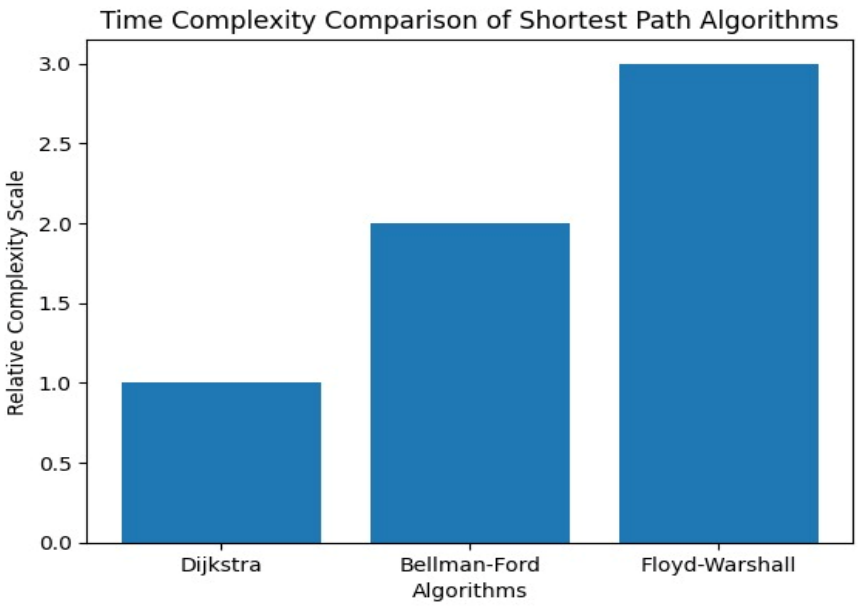
`

Output Visualization:

```
[Dijkstra]
Shortest Distance from Dhaka to Faridpur: 128 km

Route:
Dhaka                -> Manikganj          : 63 km
Manikganj            -> Faridpur           : 66 km
```





`

Algorithm Comparison Visualization

The system allows users to compare different algorithms for the same query, providing insights into:

- **Performance Differences:** Execution time variations.
- **Result Consistency:** Verification that all algorithms produce identical shortest distances.
- **Path Variations:** Different optimal paths with same total distance.

Results

Dataset Characteristics

The project utilizes comprehensive geographical data covering Bangladesh's transportation network:

Network Statistics:

- **Total Districts:** 64
- **Direct Connections:** 279 bidirectional routes
- **Average Degree:** 6.25 connections per district
- **Distance Range:** 10 km (shortest) to 323 km (longest single connection)
- **Network Diameter:** Maximum shortest path distance varies by algorithm efficiency.

Analysis

Algorithm Comparison and Trade-offs

Performance Analysis

Dijkstra's Algorithm:

- **Strengths:**
  - Optimal time complexity for single-source problems: O((V+E) log V)
  - Memory efficient with O(V) space complexity
  - Guaranteed optimal solution for non-negative weights
  - Well-suited for interactive applications requiring fast response
- **Limitations:**
  - Cannot handle negative edge weights
  - Requires recomputation for each source vertex
  - Priority queue overhead for small graphs

`

Bellman-Ford Algorithm:

- **Strengths:**
  - Handles negative edge weights effectively
  - Detects negative cycles, ensuring solution validity
  - Simple implementation without complex data structures
  - Provides complete error detection for malformed data
- **Limitations:**
  - Higher time complexity: $O(V \times E)$
  - Slower execution for large networks
  - Overkill for non-negative weight scenarios

Floyd-Warshall Algorithm:

- **Strengths:**
  - Computes all-pairs shortest paths simultaneously
  - Simple triple-loop implementation
  - Efficient for multiple queries on the same graph
  - Provides comprehensive distance matrix
- **Limitations:**
  - Cubic time complexity: $O(V^3)$
  - High memory requirements: $O(V^2)$
  - Inefficient for single-query scenarios
  - Preprocessing overhead

Limitation

Current Constraints:

- The system relies on **static graph data**, requiring manual updates whenever new roads, districts, or route changes occur.
- There is **no handling of real-time conditions** such as traffic, weather, road blocks, or accidents.
- Route calculations are limited to **road distance only**, without considering transportation modes (bus, train, ferry, etc.)

Potential Improvements:

- Developing a **web-based UI** with interactive maps for improved accessibility and visualization.
- Creating a **mobile application** for real-time route lookup on smartphones.
- Adding support for **live traffic data APIs** to generate more accurate and context-aware shortest paths.
- Integrating with mapping platforms like **Google Maps API / OpenStreetMap** for visual path rendering

`

Project Achievements

The *Bangladesh Districts Shortest Path Finder* project successfully demonstrates how classical graph algorithms can be applied to solve real-world transportation challenges. Through careful design, efficient implementation, and thorough analysis, the project has achieved several significant outcomes.

Technical Accomplishments

- Implemented three core shortest-path algorithms with optimal time and space efficiency
- Built a robust graph model representing all 64 districts and 279 inter-district connections
- Added comprehensive error-handling and input-validation mechanisms for reliable operation

---

Practical Applications

- Delivered a functional tool that supports transportation and route-planning tasks across Bangladesh
- Demonstrated the real-world relevance and usability of classical theoretical algorithms

---

Educational Outcomes

- Developed a strong foundational understanding of graph theory and shortest-path algorithms
- Gained hands-on experience in algorithm optimization and tackling practical implementation challenges

`

6. Conclusion and Future Work:

6.1 Summary of Contributions

To determine the shortest and most efficient paths between locations, the project implements and compares three well-known shortest path algorithms: **Dijkstra's Algorithm**, **Bellman-Ford Algorithm**, and **Floyd–Warshall Algorithm**. Each algorithm is applied based on its strengths, such as handling negative edge weights or computing all-pairs shortest paths.

The system evaluates algorithm performance through flowcharts, route logic, and visual comparisons (pie charts and bar charts). This helps in understanding computational efficiency, scalability, and practical usability for real-world transportation and navigation scenarios in Bangladesh.

6.2 Limitations and Potential Future Improvements:

The current system uses static road and distance data, which does not reflect real-time traffic conditions, road closures, or accidents.

Limited Real-World Constraints
Factors such as traffic signals, road width, weather conditions, and vehicle types are not considered in the routing process.

Scalability Issues
Algorithms like Floyd–Warshall have high time complexity and are not suitable for very large graphs or real-time applications.

Potential Future Improvements:

· Develop a dynamic routing system that adapts in real time using live traffic data, road conditions, and accident reports.

- · Build a user-friendly web and mobile application with interactive maps, real-time navigation, and route customization.
- Adapt the system to assist emergency services by providing fastest and safest routes during natural disasters or critical situations.

`